

Fundamentals of API Development

DevOps in API Development: Overview

Day 4 - Session 1

Day 3 Recap

From yesterday	Feeds today
A non-breaking version strategy for the shared API	Today's pipeline needs a stable versioning story to automate around
Basic security rules (authN, authZ, rate limiting)	Today's test suite needs to cover the rules you already added
The OWASP (Open Worldwide Application Security Project) API Security Top 10 framing	Today's "security within the ecosystem" segment builds on this directly

What We'll Cover

1. API-first strategy: the spec becomes the gate
2. The CI/CD pipeline, stage by stage
3. Surefire vs Failsafe: fail fast
4. DORA (DevOps Research and Assessment) metrics, observability, and "you build it, you run it"
5. Demo: gating a release
6. Demo: ordering test phases

API-First: The Spec You Already Wrote

From documentation to pipeline contract

Why API-First, in Numbers

- Schema-first teams ship roughly 2.8x faster, with 57% fewer integration defects, than code-first teams
- 82% of organizations report some level of API-first adoption; 25% are fully API-first
- Strong lifecycle discipline (naming, versioning, auth, docs, defined up front) is what keeps API-first from collapsing under delivery pressure

NOTE

None of this works without the lifecycle discipline you already practiced on Days 2 and 3.

Why 2.8x Faster?

Schema-first teams ship roughly 2.8x faster with 57% fewer integration defects.

What is the primary reason for this speed difference?

- A) Schema-first teams write better code
- B) Frontend and backend can build in parallel against the same contract
- C) Schema-first teams skip writing tests
- D) The spec auto-generates all the business logic

Chat Check

Type your answer (A, B, C, or D) in the chat window now!

Why 2.8x Faster: The Answer

B, Parallelism is the practical payoff.

Once a contract is agreed, frontend and backend stop waiting on each other. Frontend builds against mocked responses from the spec; backend implements the real logic against that same spec.

That's the same contract-first argument from Day 2, now backed by pipeline numbers.

The Spec Becomes the Gate

Once a pipeline exists, the OpenAPI spec you wrote in Lab 1.2 and peer-reviewed in Lab 1.3 becomes more than documentation:

- It drives generated server stubs and client SDKs
- It is what a contract test checks the running API against
- It is what today's pipeline validates before anything deploys

The spec you already built is what a real pipeline gates on.

The Spec Becomes the Gate (Fintech)

REAL-WORLD SCENARIO

Fintech (Navy Federal — Digital Integration):

- Navy Federal uses API-first contracts to standardize communication between their mobile app and core banking systems.
- Their CI/CD pipeline automatically blocks any build where the implementation drifts from the agreed-upon OpenAPI spec, preventing breaking changes from reaching members.

The Spec Becomes the Gate (Ecommerce)

REAL-WORLD SCENARIO

Ecommerce (Shopify — App Platform):

- Shopify strictly gates deployments on OpenAPI and GraphQL schema validations before any release to production.
- This automated enforcement ensures that thousands of third-party merchant apps never experience unexpected structural changes to the APIs they depend on.

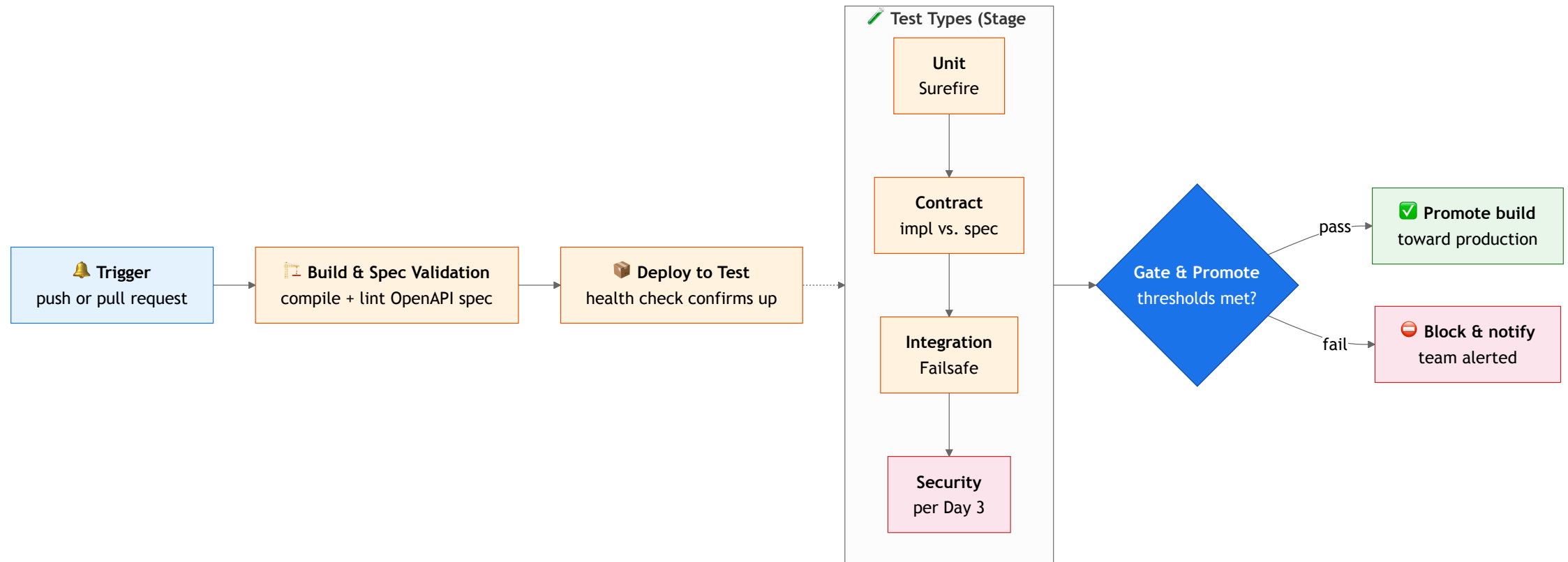
The CI/CD Pipeline, Stage by Stage

Same four stages, every time

Four Stages, Every Time

1. **Trigger** - a push or pull request starts the pipeline
2. **Build & spec validation** - compile the app, then lint the OpenAPI spec itself
3. **Deploy to test & test** - health check, then unit, contract, integration, and security tests
4. **Gate & promote** - thresholds met promotes the build; a miss blocks it and notifies the team

The Pipeline, Visualized



CI or CD? Discussion

A team merges a change. It compiles, unit tests pass, contract tests pass. Nobody deploys it for two weeks; a person clicks a button when they're ready.

Is that Continuous Integration, Continuous Deployment, both, or neither?

DISCUSS

What specific question does each half of "CI/CD" answer?

CI or CD? The Answer

	Continuous Integration	Continuous Deployment
Question	Does this change build and pass tests?	Is this build safe to ship, automatically?
API gate	Spec lint, unit tests, contract tests	Staging smoke tests, canary rollout, rollback plan
If skipped	Contract drift found late, in production	Manual, risky, infrequent releases

That scenario is CI without CD: a human still decides when to ship.

Fail Fast: Surefire vs Failsafe

Cheap tests first, expensive tests second

Two Plugins, Two Phases

	Surefire	Failsafe
Phase	test	verify
Runs	*Test classes, no external deps	*IT classes, real Spring context
Speed	Seconds	Tens of seconds or more

TIP

A broken build fails in seconds, not tens of seconds - the pipeline never pays for the expensive suite once the cheap one already caught the problem.

Maven Configuration: Surefire & Failsafe

```
<build>
  <plugins>
    <!-- Runs fast unit tests (*Test.java) during 'test' phase -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.2</version>
    </plugin>
    <!-- Runs slow integration tests (*IT.java) during 'verify' phase -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-failsafe-plugin</artifactId>
      <version>3.5.2</version>
      <executions>
        <execution>
          <goals><goal>integration-test</goal><goal>verify</goal></goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Own What You Ship

Measuring the pipeline, then staffing it

DORA Metrics: The Four Keys

DORA: DevOps Research and Assessment

Metric	2026 elite benchmark
Deployment frequency	Multiple times per day (elite: per hour)
Lead time for changes	Under 1 hour
Change failure rate	0-5%
Time to restore	Under 1 hour

Deployment frequency is the most visible signal of a healthy pipeline.

You Build It, You Run It

The team that writes an API's code also deploys it, monitors it, and gets paged for it. This collapses the traditional dev-to-ops handoff.

- **Observability** needs logs, metrics, and traces to share one correlation ID
- **Documentation-as-code**: docs live in version control next to the implementation, so they age with the code instead of drifting from it

You Build It, You Run It (Fintech)

REAL-WORLD SCENARIO

Fintech (Capital One — SRE Transformation):

- Capital One developers own their microservices end-to-end, taking on-call rotations for the specific APIs they build.
- This ownership collapses the dev-to-ops handoff, empowering the engineers who wrote the code to use correlated tracing to debug transaction failures in production.

Note: **SRE** stands for Site Reliability Engineering.

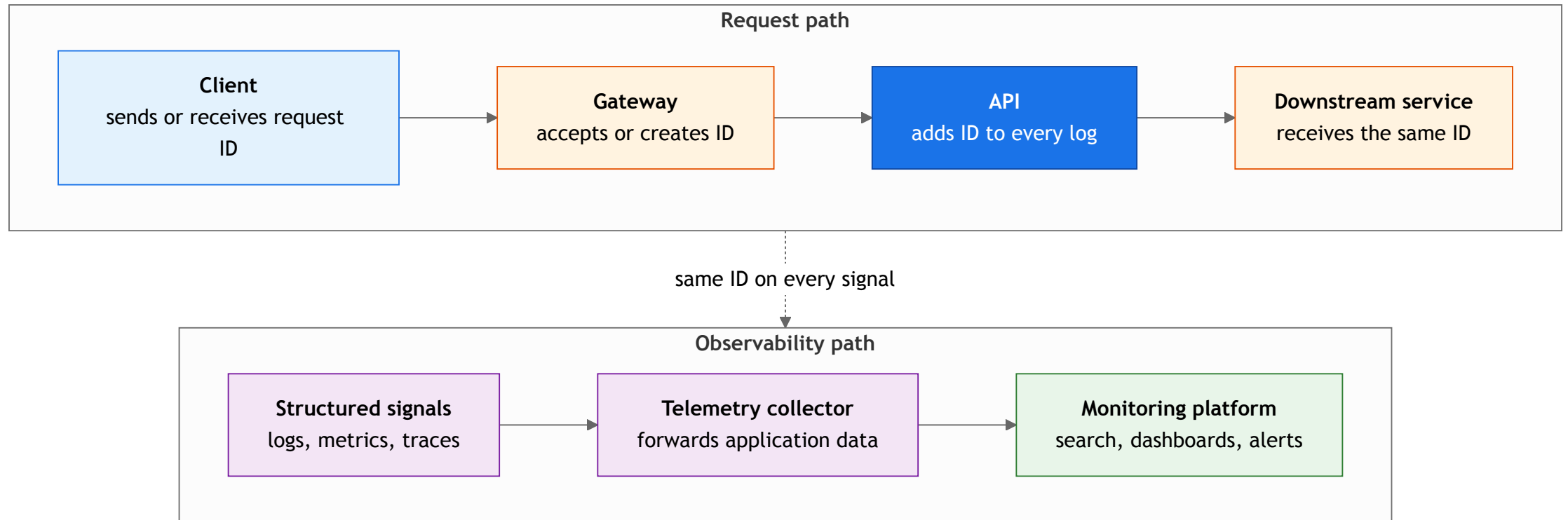
You Build It, You Run It (Ecommerce)

REAL-WORLD SCENARIO

Ecommerce (Etsy — Peak Scale Operations):

- Etsy's engineering teams use distributed observability tools to monitor their own services during massive scale events like Cyber Monday.
- When an anomaly occurs, the team responsible for the checkout API can instantly isolate the issue using correlation IDs, rather than escalating to a centralized operations desk.

One Request, One Correlation ID



Accept an incoming ID or create one, propagate it downstream, include it in logs, and return it to the caller.

From Application Signals to Monitoring

1. The application emits structured logs, metrics, and traces without tokens, passwords, or secrets
2. A collector or platform agent forwards that telemetry
3. The monitoring platform indexes it for search, dashboards, and alerts
4. During an incident, the correlation ID connects one request's events across services

The products vary; the vendor-neutral flow stays the same.

You Build It, You Run It

What is the biggest challenge your team would face if developers also had to deploy, monitor, and get paged for the APIs they write?

Chat Waterfall

Type your thoughts in the chat, but DO NOT hit enter yet.

Wait for my countdown: 3... 2... 1... BLAST

You Build It, You Run It: Key Points

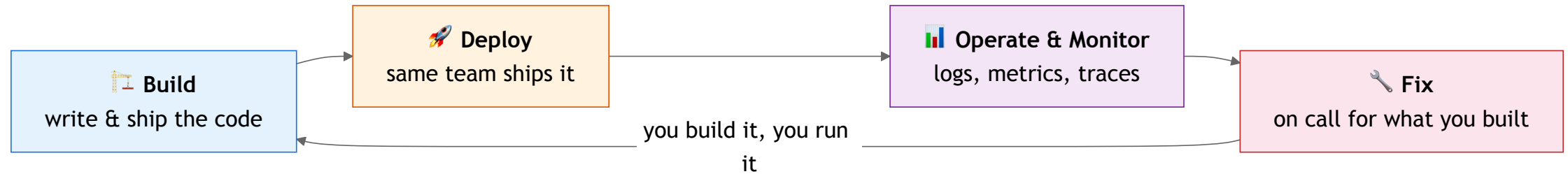
If you mentioned any of these, you're thinking like an SRE:

- **Monitoring gaps**, devs may not know what to alert on
- **Context switching**, coding vs incident response
- **Blast radius**, one team owning too many services
- **Cultural shift**, "not my problem" becomes "I built it, I own it"
- **On-call rotation**, who gets paged at 3 AM?

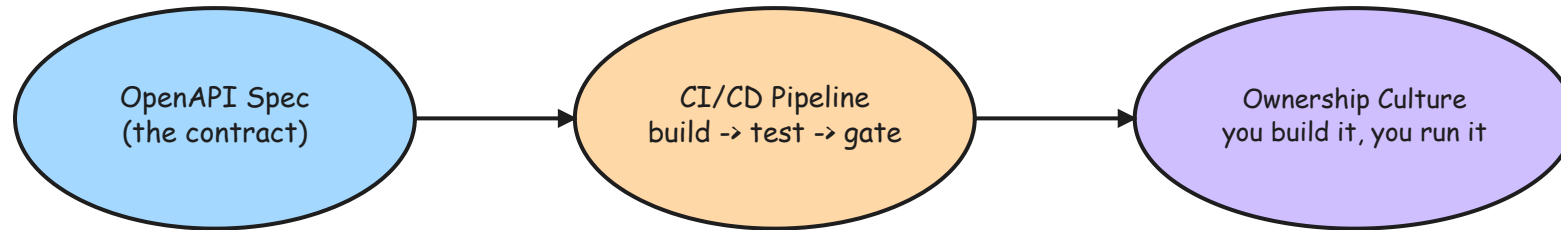
 **Pat on the back if you named any of these**

Note: **SRE** stands for Site Reliability Engineer.

The Ownership Loop



The Whole Loop, At a Glance



The spec you already wrote is what the pipeline gates on — gating is a culture, not just tooling

Demo: Pipeline Gate Simulator

Run `java -cp target/classes com.kavinschool.demos.DemoPipelineGateSimulator`
from `day4/demos`.

- Six sample builds, each carrying spec-valid, unit-test, coverage, and contract-test signals
- Each is evaluated against one promotion gate and printed as `PROMOTE` or `BLOCK`, with the reason
- A broken OpenAPI spec or a failing contract test blocks promotion exactly as surely as a failing unit test does

Offline, no API key, no network.

Demo: Surefire vs Failsafe Order

Run `java -cp target/classes com.kavinschool.demos.DemoSurefireVsFailsafeOrder` from `day4/demos`.

- Scenario 1: a broken unit test fails in the `test` phase; the pipeline stops there
- Scenario 2: unit tests pass, so the `verify` phase's integration tests run too
- A broken build fails in ~4 seconds instead of ~51 – the pipeline never pays for the expensive suite

Offline, no API key, no network.